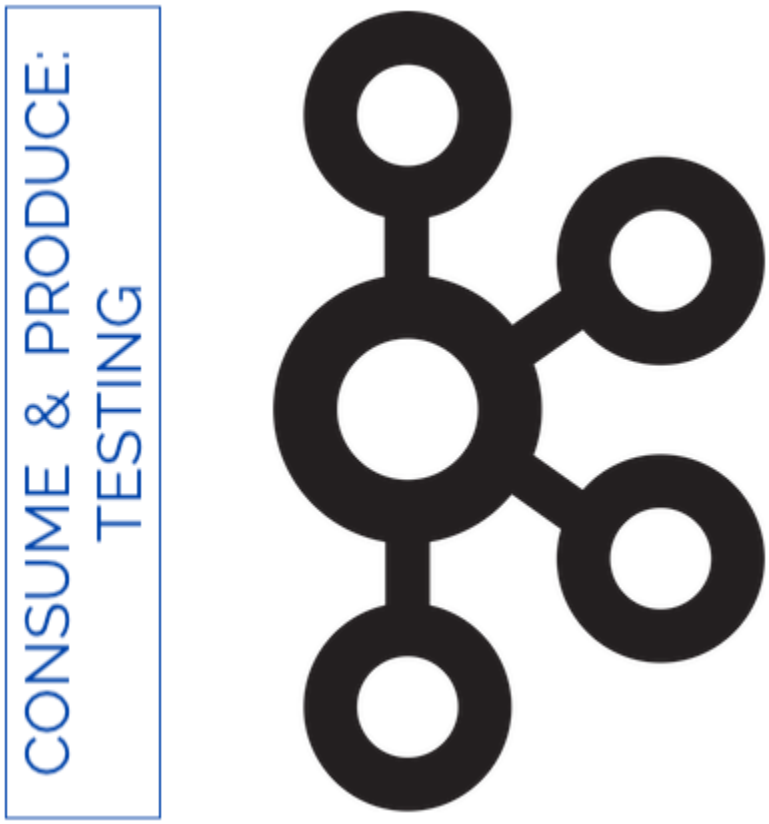
Kafka Consume & Produce: Testing

Rob Golder - July 2022



Introduction

When developing an application there are a number of types of tests that a developer should write to validate and assert that the behaviour of the application is as expected. These are unit, integration, and component tests. Each of these types verify the application in different ways. The combination of the full suite of these tests gives strong guarantees on the correctness of the application, before it reaches QA. This same suite of tests is run both locally by the developer as they develop the application, and also automated as part of the CI pipeline.

This article looks at the tools and frameworks available to a developer looking to test a Kafka application that consumes and produces messages. The companion Spring Boot application, detailed in the Kafka Consumer & Produce: Spring Boot Demo article,

demonstrates the testing techniques covered in this article. The full source code is available here:

https://github.com/lydtechconsulting/kafka-springboot-consume-produce/tree/v1.0.0

Unit Tests

Overview

Unit tests are the finest grained test, and assert the paths through a unit of code (typically a method or the methods within a class), behave as expected. Dependencies to the class under test are mocked, and calls to the mocks are verified to be as expected. For unit testing the consume and produce code there is nothing to distinguish the approach from unit testing any other code.

In the companion Spring Boot application the testing framework Mockito is used to mock the dependencies of the consumer and the producer code. This makes it straightforward to assert that these mocks have been called as expected. It is important to test all the exception flows too, to ensure that the unhappy path behaviour is as expected.

Example Tests

For unit testing the Kafka consumer then, the DemoService is mocked. The test asserts the happy path consume verifying that the mock service was called:

```
verify(serviceMock, times(1)).process(key, testEvent);
```

It also tests the unhappy path where the DemoService is mocked to throw an exception. The tests can be seen in KafkaDemoConsumerTest.

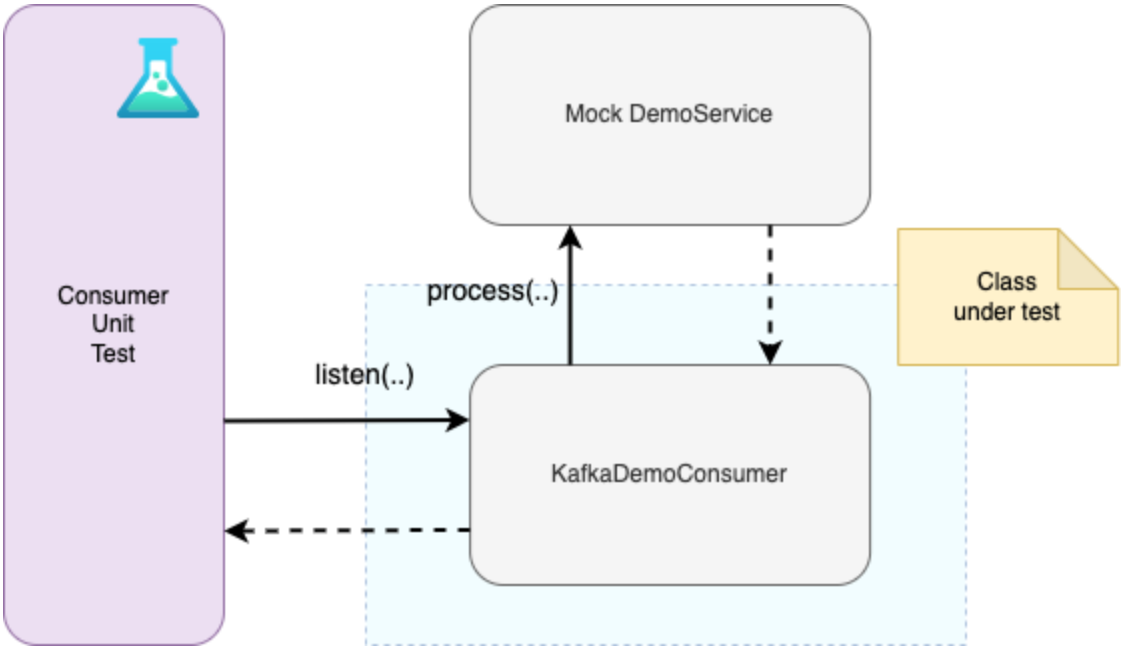


Figure 1: Unit testing the consume

For unit testing the produce, the KafkaTemplate Spring bean is mocked. KafkaTemplate is Spring Kafka's abstraction over the Kafka Producer API that makes it straightforward to send messages to Kafka. The test verifies that when the KafkaClient is called the result is a call to the KafkaTemplate.

```
verify(kafkaTemplateMock, times(1)).send(expectedRecord);
assertThat(result, equalTo(expectedSendResult));
```

The unhappy path is also tested to assert that the behaviour is as expected if the KafkaTemplate call results in an exception being thrown. The tests can be seen in KafkaClientTest.

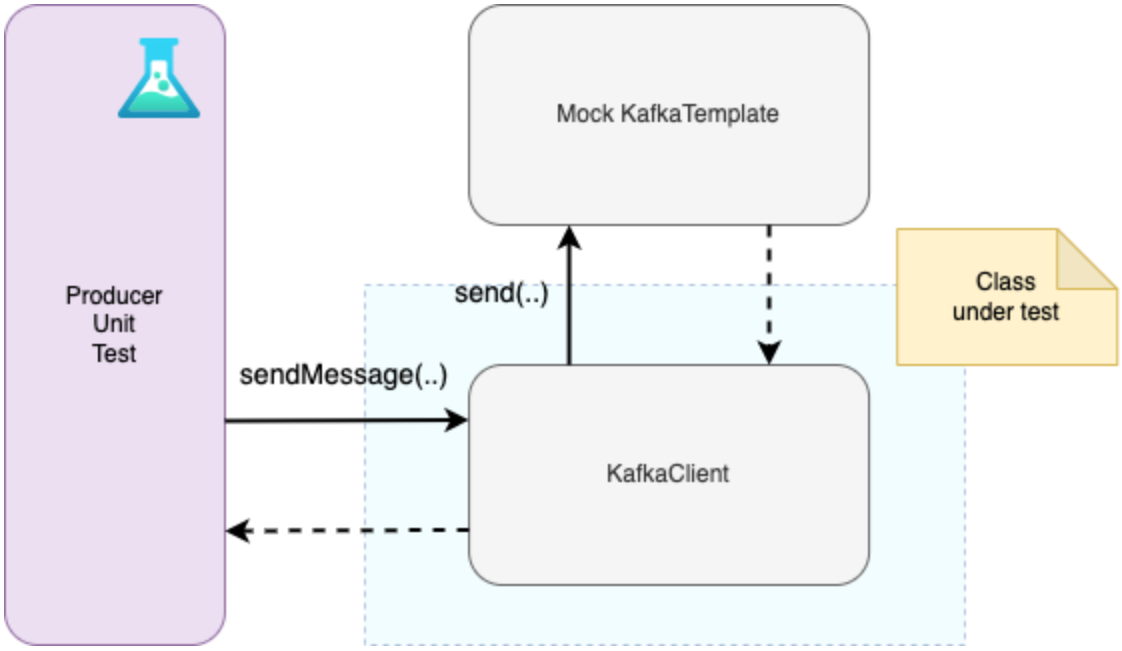


Figure 2: Unit testing the produce

Test Feedback

Unit tests provide the fastest feedback to the developer as to whether their code is correct. Typically a failing test will be written first to prove that the new or updated code results in the test passing.

Integration Tests

Overview

Integration tests are coarser grained than unit tests, and with an application that consumes and produces messages the entry and exit points of the tests will be the Kafka topics. Spring Boot tests are used that load the Spring application context, and this verifies that everything is wired together and configured as expected. In place of a real Kafka broker, the spring-kafka module provides an in-memory Kafka broker. This is achieved by changing the bootstrap server address in the integration test application properties that is picked up at test time. Both the integration tests themselves and the application will be consuming and producing from this in-memory broker.

Whilst not verifying the application against a real Kafka broker, using the in-memory broker proves that the application is correctly configured, and correctly using the Kafka consume and produce APIs to integrate with a broker. The in-memory broker can be fully configured too, so enables experimental testing, such as changing the number of broker nodes or the

number of partitions that topics are created with, and verifying the application still behaves as required.

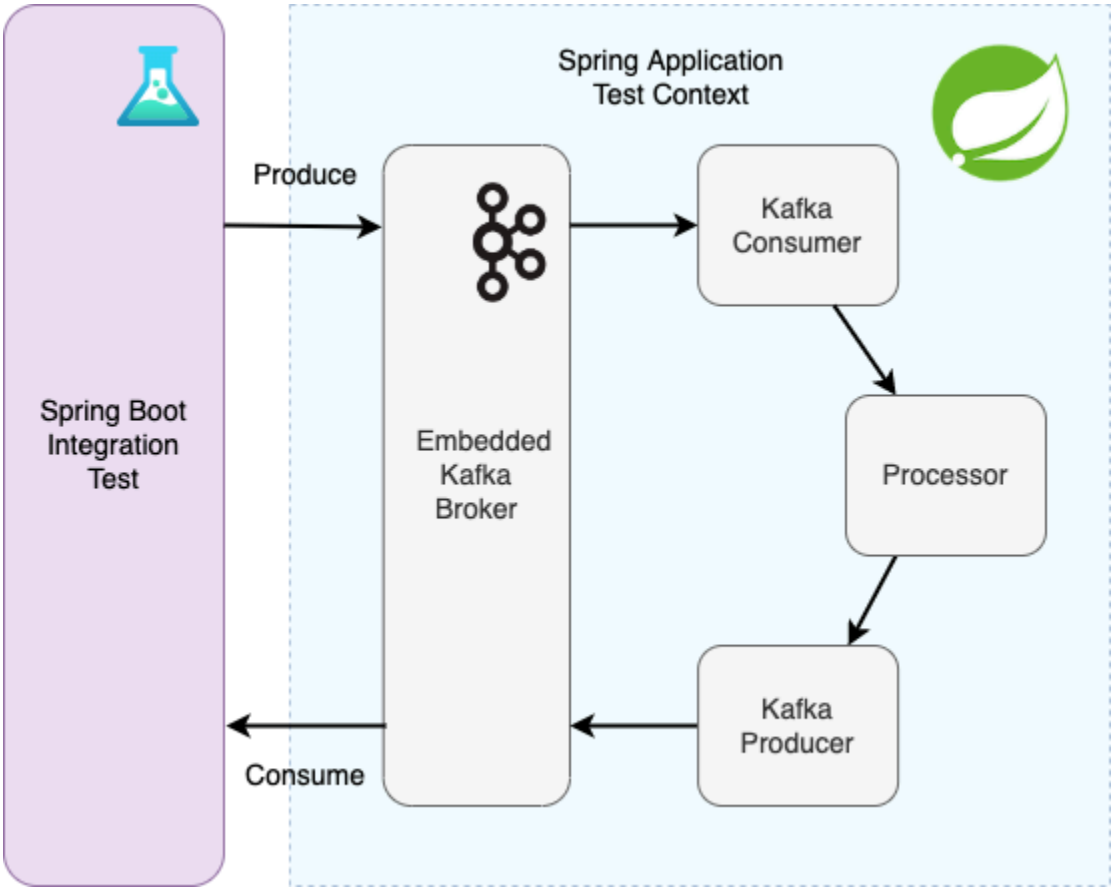


Figure 3: Integration testing

Example Test

The example Spring Boot integration test is KafkaIntegrationTest. Importantly the test is annotated with @SpringBootTest, and @EmbeddedBroker. The embedded broker annotation is all that is required to start this when the test is executed. It can be configured via parameters on the annotation.

```
@SpringBootTest(classes = { KafkaDemoConfiguration.class } )
@DirtiesContext(classMode = DirtiesContext.ClassMode.AFTER_CLASS)
@ActiveProfiles("test")
@EmbeddedKafka(controlledShutdown = true, topics = { "demo-inbound-topic"
})
```

```
public class KafkaIntegrationTest {
```

The test has the KafkaTemplate autowired in, so is able to use this to send multiple messages to the topic in the in-memory broker. It then awaits, using the Awaitility library until the expected outbound messages have been emitted by the application.

```
final SendResult result = (SendResult)kafkaTemplate.send(record).get();
Awaitility.await().atMost(10, TimeUnit.SECONDS).pollDelay(100,
TimeUnit.MILLISECONDS).until(testReceiver.counter::get,
equalTo(totalMessages));
```

To achieve this the test has its own consumer configured that is listening on the outbound topic.

```
public static class KafkaTestListener {
   AtomicInteger counter = new AtomicInteger(0);

   @KafkaListener(groupId = "KafkaIntegrationTest", topics =
"demo-outbound-topic", autoStartup = "true")
   void receive(@Payload final String payload) {
       log.debug("KafkaTestListener - Received message: " + payload);
       counter.incrementAndGet();
   }
}
```

In the example project they are run via the maven surefire plugin as part of the test phase.

If the application also made calls to other resources then these would need to be managed too. For databases an in-memory version can be swapped in, such as H2 or embedded Postgres. For calls to external services then these can be mocked with wiremock.

Where appropriate integration tests should cover unhappy paths too. For example, if processing a message results in a call to a third party service that is mocked to be not available (by returning a 503 - Service Unavailable), does the application handle this flow as required.

Test Feedback

Integration tests provide fast feedback to the developer as they are run locally on their machine (as well as in the pipeline), and do not have the overhead of spinning up containers for their dependencies.

Component Tests

Overview

Component tests are the coarsest grained test. They treat the application as a black box, proving that it can be deployed, started up, and interacted with correctly. The service is built and started in a Docker container, and a real Kafka broker instance is also started in a Docker container. The component test writes messages to Kafka, and consumes the messages emitted by the application to test for correctness.

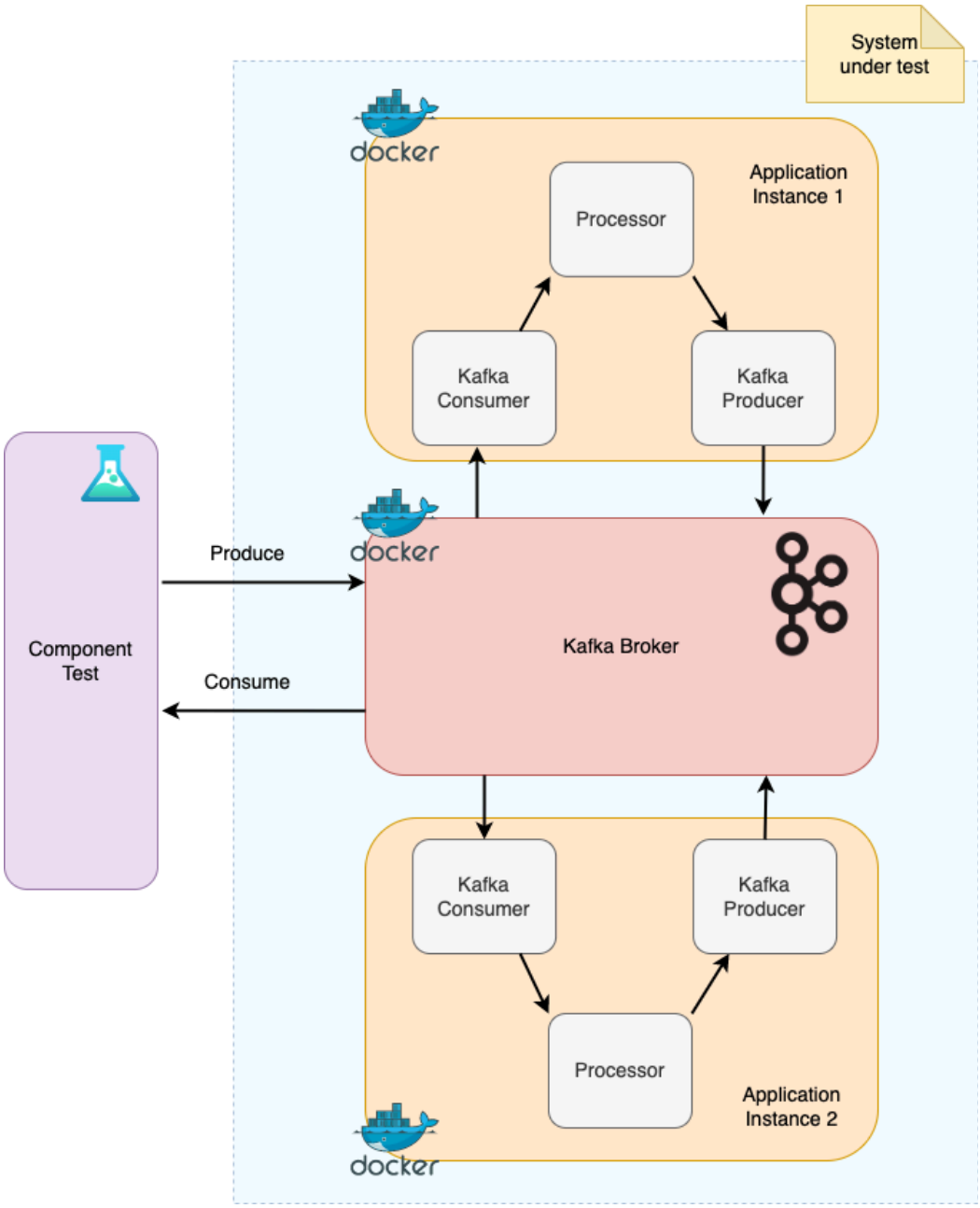


Figure 4: Component testing

Multiple instances of the service can be started up, each in their own Docker container (as shown in the diagram above). This proves that the application behaves as expected in this scenario. It can also be used for experimental testing. For example, add a remote debug breakpoint to a consumer for one service instance, and when the message is consumed and the breakpoint is hit, kill the Docker container. The message should be redelivered to the second consumer instance. This highlights Kafka's default at-least-once delivery guarantee, which is detailed in the article [Kafka Consume & Produce: At-Least-Once Delivery](#).

Example Test

The example component test [EndToEndCT](#) fires in many messages to the Kafka broker topic that are consumed by the application.

```
int totalMessages = 100;
for (int i=0; i<totalMessages; i++) {
   String key = UUID.randomUUID().toString();
   String payload = UUID.randomUUID().toString();
   KafkaClient.getInstance().sendMessage("demo-inbound-topic", key,
JsonMapper.writeToJson(buildDemoInboundEvent(payload)));
}
```

The test awaits for the outbound messages to be produced by the application via a test consumer. This asserts that the expected number of messages have been received.

```
List> outboundEvents =
KafkaClient.getInstance().consumeAndAssert("testFlow", consumer,
totalMessages, 3);
outboundEvents.stream().forEach(outboundEvent ->
assertThat(outboundEvent.value(), containsString(INBOUND_DATA)));
```

Component Test Framework

The test uses Lydtech's [component-test-framework](#) to manage the Docker containers that are required, and how they are configured. The framework uses [Testcontainers](#) to create and clean up these containers. The component-test-framework [ReadMe](#) has all the details as to how to use the framework. The key elements to use this framework with the example application are the following:

1. Add the component-test-framework dependency to the pom.xml:

```
<dependency>
```

```
  <groupId>dev.lydtech</groupId>à
  <artifactId>component-test-framework</artifactId>
  <version>1.6.0</version>
  <scope>test</scope>
```

2. `</dependency>`
   Provide a maven pom profile that defines a maven-surefire-plugin. This is then configured with the necessary properties to stipulate two Dockerised instances of the service under test, and to start Kafka in a Docker container:

```
<systemPropertyVariables>
    <service.name>${project.name}</service.name>
    <service.instance.count>2</service.instance.count>

<service.container.logging.enabled>false</service.container.logging.enabled>
    <kafka.enabled>true</kafka.enabled>

<kafka.container.logging.enabled>false</kafka.container.logging.enabled>
```

3. `</systemPropertyVariables>`
   See the full maven profile in the pom.xml. The containers can optionally be left up in between test runs while tests are being developed. See the ReadMe for details on this.
   Use the JUnit5 @ExtendWith annotation to mark the test as a component test:

```
@ExtendWith(TestContainersSetupExtension.class)
```

4. `public class EndToEndCT {`
   The TestContainersSetupExtension is provided by the component-test-framework. Build the Spring Boot application, build the Docker container via the Dockerfile, and run the component test with the -Pcomponent profile (which correlates with the profile defined in the pom.xml):

```
mvn clean install
docker build -t ct/kafka-springboot-consume-produce:latest .
mvn test -Pcomponent
```

Test Feedback

As with unit and integration tests, component tests are run locally, as well as in the pipeline. The feedback loop is the slowest of the three test types, as the service must be fully built and dockerised, and containers brought up for this and its dependencies. But it still provides

much faster feedback than waiting for the service to be deployed to a remote environment and running tests against that.

Conclusion

An application that is consuming messages from Kafka, and writing messages to Kafka, can be comprehensively tested though unit, integration, and component tests. Different tools and frameworks facilitate the testing at the different levels. These tests provide fast feedback to the developer, covering different application concerns from the correctness of small pieces of code to verifying the application can be built, deployed and run locally in a container. This gives a high level of confidence in the application before it even reaches QA.

Source Code

The source code for the accompanying Spring Boot demo application is available here:

https://github.com/lydtechconsulting/kafka-springboot-consume-produce/tree/v1.0.0

More On Kafka Consume & Produce

The following accompanying articles cover the Consume & Produce flow:

- Kafka Consume & Produce: Spring Boot Demo: details the companion application.
- Kafka Consume & Produce: At-Least-Once Delivery: looks at the default at-least-once messaging guarantees provided by Kafka.